

SY7X CE REFERENCE MANUAL

cem11a02f
May 2020



©Copyright 2020, Silergy Corp.

All rights reserved.

No part of this documentation may be reproduced nor distributed in any form or by any means, graphic, electronic, or mechanical, including but not limited to photocopying, scanning, recording, taping, e-mailing, or storing in information storage and retrieval systems without the written permission of Silergy Corp. (hereafter, “Silergy”). Products that are referenced in this document such as Microsoft Windows® may be trademarks and/or registered trademarks of their respective owners. Silergy makes no claim to these trademarks. While every precaution has been taken in the preparation of this document, individually, as a series, in whole, or in part, Silergy, the publisher, and the author assume no responsibility for errors or omissions, including any damages resulting from the express or implied application of information contained in this document or from the use of products, services, or programs that may accompany it. In no event shall Silergy, publishers, authors, or editors of this guide be liable for any loss of profit or any other commercial damage caused or alleged to have been caused directly or indirectly by this document.

Document version 2.2, May 2020

CONTENTS

1	Introduction	8
1.1	General	8
1.2	Purpose	8
2	Supported Functions	9
2.1	Data Acquisition	9
2.2	Voltage Sensor on VA	10
2.3	Line Current Sensor on IA	10
2.4	Neutral Current Sensor on IB	12
2.5	CE Busy	15
2.6	XFER Busy	15
2.7	VPulse	15
2.8	WPulse	15
2.9	XPulse	15
2.10	YPulse	15
2.11	FIRDone	15
2.12	Sync	15
2.13	Pulse Outputs	15
2.14	Device Temperature	18
2.15	Line Frequency	19
2.16	Energy & Power	20
2.17	RMS Currents	20
2.18	Three Current Vector Sum	21
2.19	RMS Voltage	21
2.20	Sensor Selection	22
2.21	SAG Detection	22
2.22	Harmonic Analysis	23
2.23	Voltage Phase Angle	24
2.24	DC Measurements	24
2.25	Chip ID	24
2.26	VLS Mode	24
3	CE Map	25
3.1	Input Parameter Map	25
3.2	Measurement Output Map	27
4	Calculation Constants	29
4.1	Sample Structure	29
5	Setting the MPU Processor	31
5.1	MPU	31
5.2	The CE Init and Start Function	34
6	CE Resource Usage	39
6.1	CE Code Flash Size	39
6.2	CE Data Flash Size	39
6.3	CE Cycles Per Pass	39

7	CE Operational Considerations	40
7.1	CE Validation at Power Up	40
7.2	Filtering Out Unwanted Pulses At Start Up	40
8	Example Calculations	41
8.1	RMS Voltage	41
8.2	RMS Current	41
8.3	Energy & Power	42
8.4	SAG Threshold	42
9	Using QUANT to Compensate for Nonlinearities	43
9.1	ENERGY	43
9.2	CURRENT	43
9.3	VOLTAGE	44

TABLES

Table 1	Meter Equations Active/Reactive Elements	9
Table 2	Meter Equations Voltage/Current Elements	10
Table 3	Voltage Sensor Inputs	10
Table 4	Current Inputs	10
Table 5	Current Inputs	12
Table 6	Pulse Configuration - Inputs	16
Table 7	Pulse Outputs	16
Table 8	Pulse Configuration	17
Table 9.	Device Temperature Inputs	18
Table 10.	Device Temperature Outputs	18
Table 11.	Device Temperature External Inputs	19
Table 12	Line Frequency Configuration Inputs	19
Table 13	Quant Compensation For Energy & Power - Inputs	20
Table 14	Energy & Power - Outputs	20
Table 15	Quant Compensation For Current - Inputs	20
Table 16	Current - Outputs	21
Table 17	Quant Compensation For Voltage - Inputs	21
Table 18	Voltages - Outputs	21
Table 19	SAG Configuration - Inputs	22
Table 20	SAG Status - Outputs	22
Table 21	NFREQ - Inputs	23
Table 22	Harmonics - Outputs	23
Table 23	Fundamental - Outputs	24
Table 24	Chip ID Outputs	24
Table 25	CE Data RAM	25
Table 26	CE RAM - Inputs	25
Table 27	CE RAM - Outputs	27

Table 28 CE Constants	29
Table 29 MPU Core Configurations for Use with two CTs or Shunt/CT	31
Table 30 MPU Core Configurations for Use with two CTs or Shunt/CT in Low Power Mode	33
Table 31 CE Constants For 1m Ω shunt	41

FIGURES

Figure 1 ADC Inputs with Shunt/CTs for 2 Wire, 1 Phase Systems	9
Figure 2 Voltage Measurement	10
Figure 3 Line Current Measurement	11
Figure 4 Determining IBSCALE and I1_SCALE Value	14
Figure 5 Nonlinearity at Low Current	43

1 Introduction

1.1 General

The Compute Engine firmware *cem11a02f* was created to address the requirements for the Single phase Metering market implemented on a SY7T108 platform. The CE aggregates environmental data from the ADCs, temperature sensor and other sources then supplies preprocessed outputs to the MPU for user consumption. The CE may also handle selected pulse outputs.

The CE operates as a slave to the AFE timing and runs a single set of instructions every time new set of ADC samples are available. This CE code is designed to run at 4096 Hz sample rate. The CE performs several basic functions on each code pass.

- Data Acquisition
- Signal Conditioning
- Line Frequency Synchronization
- Calculate Products and Sums
- Update RAM based Outputs and Status
- Update and Drive Pulse Outputs

In addition this ce code supports a low power mode that measures current only and runs at 324.44Hz.

1.2 Purpose

This document intends to describe the functional capabilities and requirements for the CE firmware in its intended environment.

2 Supported Functions

2.1 Data Acquisition

This CE firmware is intended for meter equation 0 (EQU=0) for use in **2 wire, one phase** systems connected to the SY7T M11 as shown in Figure 1 with shunt/current transformers (CT).

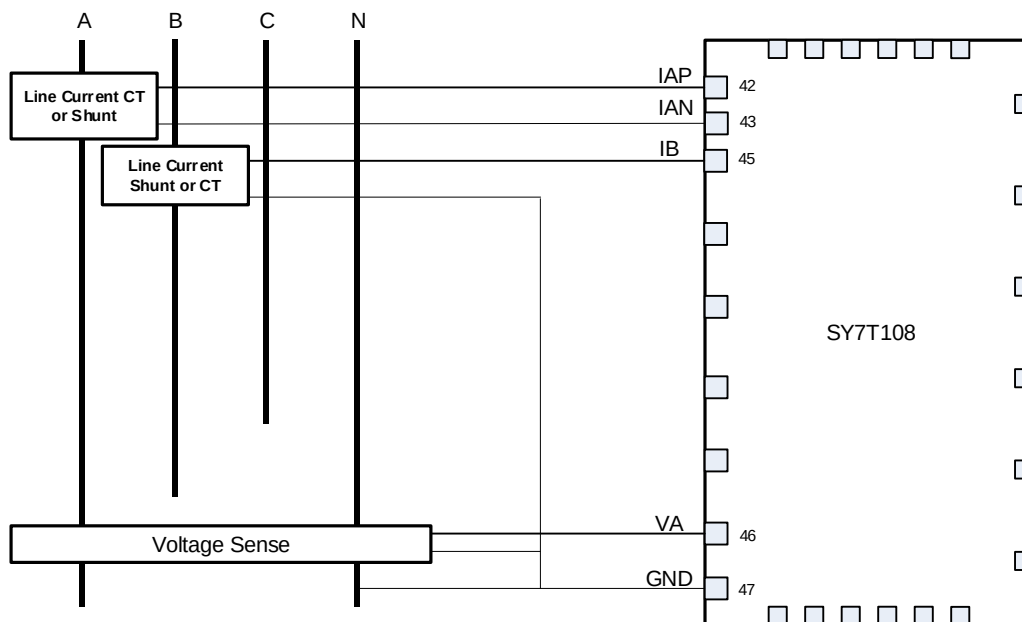


Figure 1 ADC Inputs with Shunt/CTs for 2 Wire, 1 Phase Systems

Table 1 and Table 2 shows a summary of all meter equations supported in this ce code.

Table 1 Meter Equations Active/Reactive Elements

Meter Type	Active/Reactive Equations				EQU
	WSUM/VARSUM	W0SUM/ VAR0SUM	W1SUM/VAR1SUM	W2SUM/VAR2SUM	
1p2w	V0*I0	V0*I0	V0*I1	N/A	0

Table 2 Meter Equations Voltage/Current Elements

Meter Type	VxSQSUM_X and IxSQSUM_X Inputs						EQU
	V0SQSUM_X	V1SQSUM_X	V2SQSUM_X	I0SQSUM_X	I1SQSUM_X	I2SQSUM_X	
1p2w	V0	N/A	N/A	I0	I1	N/A	0

Notes:

1p2w is single phase, 2 wire meter.

The analog to digital converters (ADC) are Sigma Delta and equivalent to 23 bits for the I0 staring ADC and 20 bits for the V0/I0 multiplexed ADC.

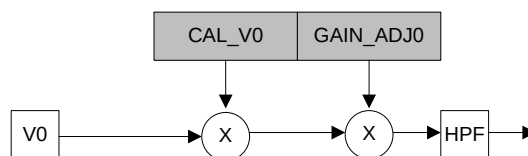
2.2 Voltage Sensor on VA

Voltage expects to be measured with a resistive divider. This input is used in the RMS line voltage as well as energy and power calculations. The divider is characterized by its full scale voltage (FSV_L) which, for the purposes of this document, we are defining as the peak voltage magnitude at which the ADC input sees 250mVpk.

Voltages are mapped internally in the CE DRAM to the following variables VA to V0.

Table 3 Voltage Sensor Inputs

Name	Offset	Note
GAIN_ADJ0	0x55	ADC gain temperature compensation curve coefficient applied to V0
CAL_V0	0x11	Calibration Gain Adjustment V0



$$V' = V * \frac{CAL_V0}{2^{14}} * \frac{GAIN_ADJ0}{2^{14}}$$

Figure 2 Voltage Measurement

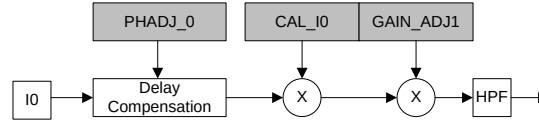
2.3 Line Current Sensor on IA

Line current is measured by shunt and CT (current transformer) sensors. This input is used in the RMS line current as well as energy and power calculations. The divider is characterized by its full scale current (FSI_L) which, for the purposes of this document, we are defining as the peak current magnitude at which the ADC input sees 250mVpk.

Currents are mapped internally in the CE DRAM to the following variables IA to I0.

Table 4 Current Inputs

Name	Offset	Note
CAL_I0	0x10	Calibration Gain Adjustment for I0
PHADJ_0	0x12	Phase Adjustment I0
GAIN_ADJ1	0x56	ADC gain temperature compensation curve coefficient for I0



$$I' = I * \frac{CAL_I0}{2^{14}} * \frac{GAIN_ADJ1}{2^{14}}$$

Figure 3 Line Current Measurement

PHADJ_0 uses CT compensation method. A CT usually generates phase lead. If it is desired to compensate by delaying current by φ radians at frequency f :

$$\begin{aligned}
 PHADJ(\varphi, f) &= \frac{2^{14}}{\alpha} \frac{\tan \varphi [1 + \beta^2 - 2\beta \cos(2\pi fT)]}{\beta \sin(2\pi fT) - \tan \varphi [1 - \beta \cos(2\pi fT)]} \\
 &= 2^{20} \frac{0.008452 \tan \varphi}{0.09172 - 0.00618 \tan \varphi} \quad \text{at } 60 \text{ Hz} \\
 &= 2^{20} \frac{0.0058722 \tan \varphi}{0.076474 - 0.004887 \tan \varphi} \quad \text{at } 50 \text{ Hz}
 \end{aligned}$$

$$|H_{PHADJ}(f)| = \sqrt{1 + \frac{\alpha \gamma [2 + \alpha \gamma - 2\beta \cos(2\pi fT)]}{1 - 2\beta \cos(2\pi fT) + \beta^2}}$$

$$\alpha = 2^{-6} \quad \beta = 1 - 2^{-9} \quad \gamma = \frac{PHADJ}{2^{14}}$$

Where T is sampling time ($1/F_s$), f is line frequency.

Minimum practical compensation corresponds to PHADJ_0= -8,816:

$$\min \varphi_{PHADJ} \approx -10.00^\circ \quad \text{at } f = 50 \text{ Hz}$$

Maximum practical compensation corresponds to PHADJ_0= 8,944:

$$\max \varphi_{PHADJ} \approx 10.00^\circ \quad \text{at } f = 50 \text{ Hz}$$

Note that PHADJ phase compensation method introduces additional gain $|H_{PHADJ}|$ to I0 signal. That additional gain should be accounted for by adjusting CAL_I0:

$$CAL_I0 = CAL_I0_{before} \frac{|H_{PHADJ}|_{before}}{|H_{PHADJ}|_{after}}$$

where subscripts 'before' and 'after' mark quantities before and after phase compensation, correspondingly.

2.4 Neutral Current Sensor on IB

When using this ce code and EQU=0 with tamper features, I0 is line current and I1 is neutral current, the neutral full scale current (I1) FSI_N will be different from line full scale current (I0) FSI_L . As result neutral current sample LSB will not match line current sample LSB. The full scale current values are based on **both** the IMAX value and current CE conversion constants because the I0 channel using staring analog to digital converter (adc) and I1 channel uses the multiplexed (adc) instead of just ratios of IMAX values. The formula for the full scale current ratio is then

$$\frac{FSI_N}{FSI_L} = \sqrt{\frac{C_{INSQ}}{C_{ISQ}}} \times \frac{IMAX1}{IMAX0}$$

In order to equalize sample LSB representation for neutral current and for line currents, variables I1_SCALE and IBSCALE should be set to a value based on the corresponding to a ratio of full scale currents as shown in Figure 4.

Currents are mapped internally in the CE DRAM to the following variables IB to I1.

Table 5 Current Inputs

Name	Offset	Note
CAL_I1	0x13	Calibration Gain Adjustment I1
PHADJ_1	0x15	Phase Adjustment I1
GAIN_ADJ2	0x57	ADC gain temperature compensation curve coefficient I1
I1_SCALE	0x16	Fine Neutral-Line Scaling Factor (x2<)
IBSCALE	0x17	Coarse Neutral-Line Scaling Factor (x1, x2, x4, x8, or x16)

PHADJ_1 uses CT compensation method. A CT usually generates phase lead. If it is desired to compensate by delaying current by φ radians at frequency f :

$$PHADJ(\varphi, f) = \frac{2^{14}}{\alpha} \frac{\tan \varphi [1 + \beta^2 - 2\beta \cos(2\pi fT)]}{\beta \sin(2\pi fT) - \tan \varphi [1 - \beta \cos(2\pi fT)]} = 2^{20} \frac{0.008452 \tan \varphi}{0.09172 - 0.00618 \tan \varphi} \quad \text{at } 60 \text{ Hz}$$

$$= 2^{20} \frac{0.0058722 \tan \varphi}{0.076474 - 0.004887 \tan \varphi} \quad \text{at } 50 \text{ Hz}$$

$$|H_{PHADJ}(f)| = \sqrt{1 + \frac{\alpha \gamma [2 + \alpha \gamma - 2\beta \cos(2\pi fT)]}{1 - 2\beta \cos(2\pi fT) + \beta^2}}$$

$$\alpha = 2^{-6} \quad \beta = 1 - 2^{-9} \quad \gamma = \frac{PHADJ}{2^{14}}$$

Where T is sampling time ($1/F_s$), f is line frequency.

Minimum practical compensation corresponds to PHADJ_1= -8,816:

$$\min \varphi_{PHADJ} \approx -10.00^\circ \text{ at } f = 50\text{Hz}$$

Maximum practical compensation corresponds to PHADJ_1= 8,944:

$$\max \varphi_{PHADJ} \approx 10.00^\circ \text{ at } f = 50\text{Hz}$$

Note that PHADJ phase compensation method introduces additional gain $|H_{PHADJ}|$ to I1 signal. That additional gain should be accounted for by adjusting CAL_I1:

$$CAL_{I1} = CAL_{I1_{before}} \frac{|H_{PHADJ}|_{before}}{|H_{PHADJ}|_{after}}$$

where subscripts 'before' and 'after' mark quantities before and after phase compensation, correspondingly.

Determining IB Scaling Variable Values

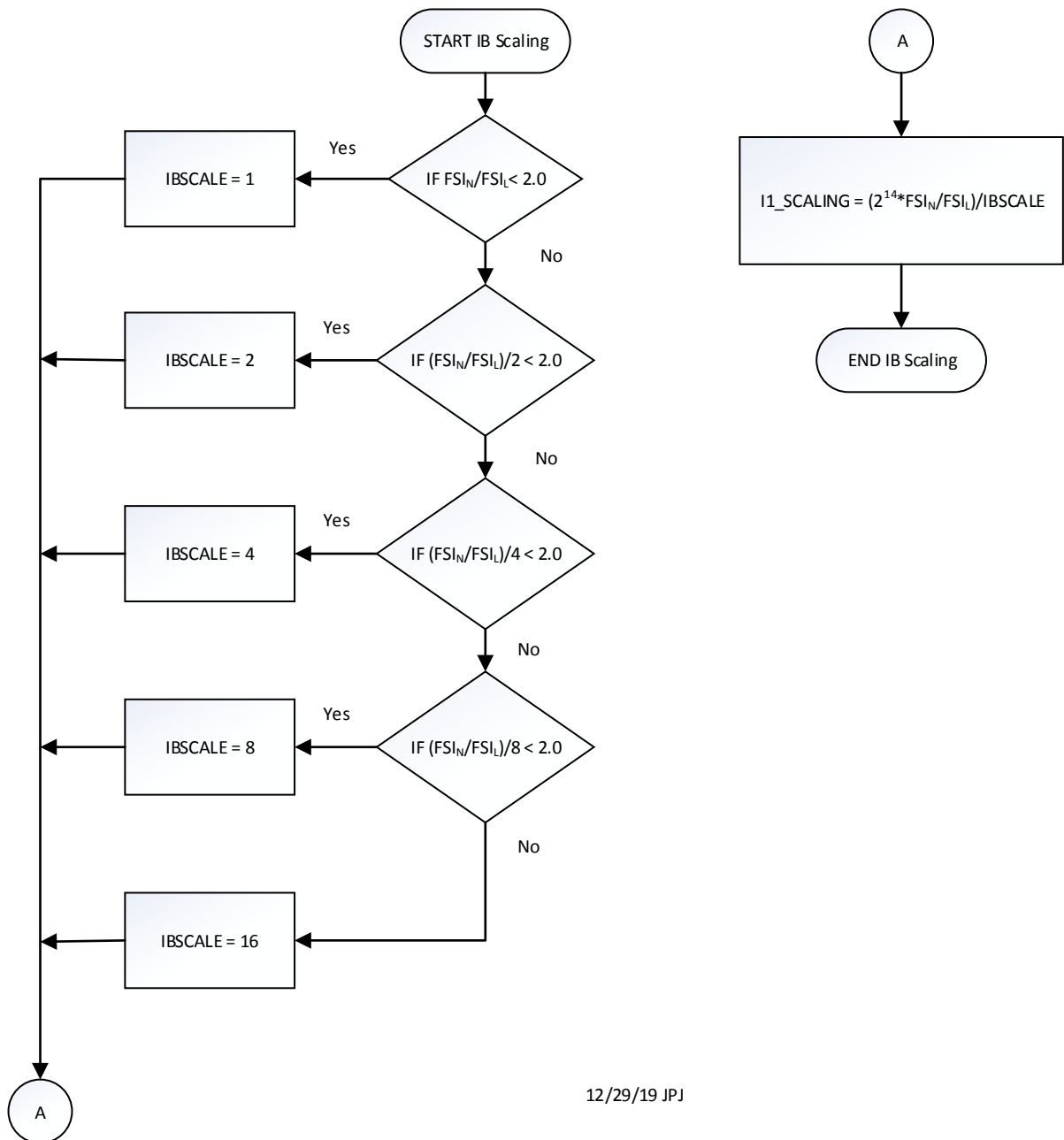
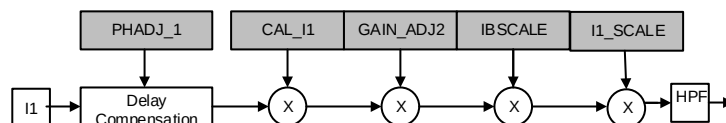


Figure 4 Determining IBSCALE and I1_SCALE Value

The value I1_SCALE must not exceed 32767 and its default value is 16384.



$$I1' = I1 * \frac{CAL_I1}{2^{14}} * \frac{GAIN_ADJ2}{2^{14}} * \frac{IBSCALE * I1_SCALE}{2^{14}}$$

Figure 5 Neutral Current Measurement

2.5 CE Busy

This interrupt is unused by the CE.

2.6 XFER Busy

The CE expects the MPU to copy and/or utilize all RAM outputs immediately after the falling edge of this interrupt.

2.7 Sync

This interrupt is unused by the CE.

2.8 VPulse

VPULSE is expected to be configured for the pulse generator output for VARh pulses. The debugger's reset (SWD_RST) and VPULSE share the same pin. See pulse outputs for details. Enabling VPULSE when the debugger is connected will cause the debugger to malfunction.

2.9 WPulse

WPULSE is expected to be configured for the pulse generator output for Wh pulses. See pulse outputs for details.

2.10XPulse

When enabled in the CE (see CECONFIG) this interrupt can be used to indicate zero-crossing of voltage waveform. This feature should be used carefully because noise generated from the signal could cause issues with low end current accuracy performance.

2.11 YPulse

When enabled in the CE (see CECONFIG) this interrupt can be used to indicate when line voltage sag is detected.

2.12 FIRDone

This interrupt is unused by the CE.

2.13Pulse Outputs

The CE code will use WPULSE to output Wh pulses based upon accumulated WSUM_X. The CE code will use APULSEW (instead of WSUM_X) if external pulse control is selected (EXT_PULSE==1). The amount of Wh energy per pulse is determined by the WRATE value.

The CE code can use VPULSE to output VARh pulses based upon accumulated VARSUM_X. The CE code will use APULSER (instead of VARSUM_X) if external pulse control is selected (EXT_PULSE==1). The amount of VARh per pulse is determined by the WRATE value.

Table 6 Pulse Configuration - Inputs

Name	Offset	Note
WRATE	0x21	Pulse rate setting
CECONFIG[1]	0x20	PULSE_FAST==1: pulses are worth 1/16 nominal value
CECONFIG[0]	0x20	PULSE_SLOW==1: pulses are worth 64x nominal value
CECONFIG[5]	0x20	EXT_PULSE==1: pulses are derived from external inputs
APULSEW	0x5A	Used for WPULSE output if EXT_PULSE==1
APULSER	0x5E	Used for VPULSE output if EXT_PULSE==1
CREEP_THR	0x25	If INT_CREEP==1, compare the input of each pulse generator (WSUM_X and VARSUM_X) to CREEP_THR and zero if below. The value of WSUM_X/VARSUM_X is dependent on meter equation EQU. CREEP_THR = W(or VAR)/(2.96696e-9 * VMAX * IMAX)
CECONFIG[3]	0x20	INT_CREEP==1: internal CREEP applied to WPULSE and VPULSE pulse generator inputs WSUM_X and VARSUM_X based on CREEP_THR and meter equation EQU.

Table 7 Pulse Outputs

Name	Offset	Note
WSUM_ACCUM	0x5B	Rollover accumulator for WPULSE
WPULSE_CTR	0x59	Counts the number of whole WPULSE
WPULSE_FRAC	0x5A	Fractional part of WPULSE
VSUM_ACCUM	0x5F	Rollover accumulator for VPULSE
RPULSE_CTR	0x5D	Counts the number of whole VPULSE
RPULSE_FRAC	0x5E	Fractional part of VPULSE

Fractional part WPULSE_FRAC interpreted as unsigned integer provides an alternative power measure represented as number of pulses:

$$\text{Number of W-Pulses} = \text{WPULSE_CTR} + \frac{\text{WPULSE_FRAC}}{2^{32}}$$

Similarly, fractional part VPULSE_FRAC interpreted as unsigned integer provides an alternative power measure represented as number of pulses:

$$\text{Number of V-Pulses} = \text{VPULSE_CTR} + \frac{\text{RPULSE_FRAC}}{2^{32}}$$

WRATE LSB is related to the WSUM_X (and other power outputs) as follows.

$$WRATE = 2^{46} * \frac{(VMAX * IMAX) * \left(\frac{C_p}{FS * NACC}\right)}{\left(\frac{watt \times sec}{pulse}\right)} * \frac{(2^{6*PULSE_SLOW}) * (2^{-4*PULSE_FAST})}{X}$$

$$\frac{watt \times sec}{pulse} = 2^{46} * \frac{(VMAX * IMAX) * \left(\frac{C_p}{FS * NACC}\right)}{WRATE} * \frac{(2^{6*PULSE_SLOW}) * (2^{-4*PULSE_FAST})}{X}$$

The Kh of the meter is related to WRATE by the following equation:

$$Kh = VMAX * IMAX * KK / (WRATE * NACC * X') \text{ Wh/pulse}$$

Where

KK = 51.3518 when used with shunt or CTs

NACC is the accumulation period determined by SUM_SAMPS

X' is pulse scaling factor determined as follows using PULSE_SLOW and PULSE_FAST from CECONFIG.

Table 8 Pulse Configuration

PULSE_FAST	PULSE_SLOW	X'
0	0	6
0	1	0.09375
1	0	96
1	1	1.5

The pulse characteristics are defined by PLS_INTERVAL (pulse sampling interval), PLS_WID_MAX (pulse width maximum), and the value of X'=6 for this CE code.

During each CE code pass, the hardware stores exported WPULSE and VPULSE sign bits in an 8 bit FIFO and outputs them at specified interval. This permits the CE code to calculate the WPULSE/VPULSE outputs at the beginning of its code pass and to rely on hardware to spread them over the MUX frame. The FIFO is reset at the beginning of each MUX frame. PLS_INTERVAL controls the delay to the first pulse and the interval between subsequent updates. The LSB of PLS_INTERVAL is equivalent to 4 PLL_SPD/2 cycles. If zero, the FIFO is deactivated and the pulse outputs are updated immediately. This NINTERVAL is 4*PLS_INTERVAL.

Since the FIFO resets at the beginning of each MUX frame, PLS_INTERVAL must be specified in each ce code to ensure that all updates are output before the MUX frame completes per the formula below.

$$PLS_INTERVAL = (Ps * (PLL_SPD/2) - 4) / (4 * n)$$

$$PLL_SPD/2 = 20.447232\text{MHz}/2 = 10.223626\text{MHz}$$

$$Ps = 1/Fs = 1/4096\text{Hz (for this ce code)}$$

$$n = \text{pulse calculations/CE code pass} = 6 \text{ (for this ce code)}$$

PLS_INTERVAL = 103.8 or 103 and if PLS_INTERVAL is set to 104, the fifth output will occur too late and be lost.

PLS_WID_MAX selects a maximum negative pulse width, NMAX, updates per MUX frame per the formula:
 $NMAX = (2 * PLS_WID_MAX + 1) * PLS_INTERVAL * 4 * 1 / (PLL_SPD / 2)$

Maxim pulse width (NMAX) = $(2 * PLS_WID_MAX + 1) * PLS_INTERVAL * 4 * 1 / (PLL_SPD / 2)$

For NMAX = 20mS (for this example)

Fs = 4096Hz (for this ce code)

PLS_WID_MAX = 247.64 or 248

If the pulse period corresponding to the pulse rate exceeds the desired pulse width a 50% duty is generated.

If PLS_WID_MAX = 0xFF, the pulsewidth is always 50%.

2.14 Device Temperature

Note: This ce code does not support internal temperature compensation as described in this section and the MPU must perform external temperature compensation with EXT_TEMP=1 in CECONFIG [bit 22] (0x20). Default value of EXT_TEMP=0.

The device has a temperature sensor in the die. The MPU is expected to configure the temperature sensor to update its output to STEMP periodically. The CE can access this reading automatically and will compensate the ADC readings accordingly.

Table 9. Device Temperature Inputs

Name	Offset	Note
PPMC	0x1A	ADC gain temperature compensation curve coefficient
PPMC2	0x1B	ADC gain temperature compensation curve coefficient
STEMP22	0x18	Set to value of STEMP_X reading at 22°C
DEGSCALE	0x19	Scaling factor to produce TEMP_X from STEMP_X readings. Default = 13213.
CECONFIG[22]	0x20	EXT_TEMP==1:GAIN_ADJ0 is not updated based upon TEMP_X

Table 10. Device Temperature Outputs

Name	Offset	Note
STEMP_X	0x91	Averaged raw temperature sensor output
TEMP_X	0x81	Scaled temperature output, (t–22°C) difference in 0.1°C units
GAIN_ADJ0	0x55	V0, V1 Gain Adjust
GAIN_ADJ1	0x56	I0 Gain Adjust
GAIN_ADJ2	0x57	I1 Gain Adjust

The MPU is expected to set the STEMP22 and DEGSCALE. Inputs such that temperature reading (TEMP_X) are as follows. The CE can automatically compensate for gain variations over device temperature. The device assumes temperature is normalized at 22°C.

$$TEMP_X = \frac{DEGCALE * (STEMP_X - STEMP22)}{2^{46}}$$

$$DeviceTemperature = 22^{\circ}C + \frac{TEMP_X}{10}$$

$$GAIN_ADJ0 = 2^{14} + \frac{PPMC * TEMP_X}{2^{14}} + \frac{PPMC2 * TEMP_X^2}{2^{20}}$$

If the EXT_TEMP bit in CECONFIG is set then the CE will not update GAIN_ADJ0 and instead rely on the MPU to calculate the proper adjustment.

The section below shows the recommended MPU usage for setting PPMC based upon external information.

Table 11. Device Temperature External Inputs

Name	Units	Note
Vref(22°C)	V	See Data Sheet for parameter details
TC1	V/°C	See Data Sheet for parameter details
TC2	V/(°C ²)	See Data Sheet for parameter details

$$PPMC = \left(\frac{TC1}{10 * V_{ref}(22^{\circ}C)} \right) 2^{28}$$

$$PPMC2 = \left(\frac{TC2}{100 * V_{ref}(22^{\circ}C)} \right) 2^{34}$$

2.15 Line Frequency

Line frequency is detected through the CE firmware within 45–55Hz range. The line frequency is calculated from the incoming voltage signal (VA). The CE will also count line voltage zero crossings (2 per cycle) to allow long term time/frequency tracking. The CE will output square wave F0 synchronized to selected V-waveform.

Table 12 Line Frequency Configuration Inputs

Name	Offset	Note
FREQ_X	0x82	Line voltage frequency computed in the CE
MAINEDGE_X	0x83	Number of Zero crossings in the accumulation interval
CECONFIG[7:6]	0x20	FREQSEL: Selects Voltage signal used for frequency measurement 00 — select V0
CESTATUS[3]	0x80	F0 : square wave synchronized to V-waveform selected by FREQSEL. Edges correspond to zero crossings.

$$\text{Line Frequency (LF)} = \text{FREQ_X} * \frac{FS}{2^{32}}$$

Where FS is the sample frequency

2.16 Energy & Power

Energy outputs are calculated by performing multiply and accumulate function on the (preprocessed) voltage and current samples over an accumulation interval. These outputs give an indication (in atypical units) of the various forms of measured energy. The MPU can then calculate the energy or power into typical user defined unit from this data.

Table 13 Quant Compensation For Energy & Power - Inputs

Name	Offset	Note
QUANT_W0	0x29	Quantization compensation
QUANT_W1	0x2D	Quantization compensation
QUANT_VAR0	0x2A	Quantization compensation
QUANT_VAR1	0x2E	Quantization compensation

Table 14 Energy & Power - Outputs

Name	Offset	Note
W0SUM_X	0x85	Sum of (Current * Voltage) Samples (I0*V0)
W1SUM_X	0x86	Sum of (Current * Voltage) Samples (I1*V0)
WSUM_X	0x84	Total accumulation = W0SUM_X
VAR0SUM_X	0x88	Sum of (Current * 90°shifted Voltage) Samples
VAR1SUM_X	0x89	Sum of (Current * 90°shifted Voltage) Samples
VARSUM_X	0x87	Total accumulation= VAR0SUM_X

$$\text{SUM_X}' = \text{SUM_X} + \frac{\text{QUANT_x} * \text{SUM_SAMPS}}{2^{14}}$$

$$\text{Power n} = (\text{VMAX} * \text{IMAX}) * \left(\frac{C_P}{\text{SUM_SAMPS}} \right) * \text{SUM_X}$$

$$\text{Energy n} = (\text{VMAX} * \text{IMAX}) * \left(\frac{C_P}{FS} \right) * \text{SUM_X}$$

2.17 RMS Currents

Table 15 Quant Compensation For Current - Inputs

QUANT_I0	0x28	Quantization compensation
QUANT_I0	0x28	Quantization compensation
QUANT_I1	0x2C	Quantization compensation

Table 16 Current - Outputs

Name	Offset	Note
I0SQSUM_X.H	0x8A	Sum of Squares of Current Samples high bits for I0
I0SQSUM_X.L	0x8B	Sum of Squares of Current Samples low bits for I0
I1SQSUM_X.H	0x8C	Sum of Squares of Current Samples high bits for I1
I1SQSUM_X.L	0x8D	Sum of Squares of Current Samples low bits for I1

$$\text{InSQSUM_X}' = \text{InSQSUM_X.H} + \frac{\text{InSQSUM_X.L}}{2^{11}} + \frac{\text{QUANT_In}}{2^{11}} * \frac{\text{SUM_SAMPS}}{2^{11}}$$

Where QUANT_In=0.

$$\text{IRMS}_n = \text{IMAX} * \sqrt{\frac{C_{ISQ} * (\text{InSQSUM_X}')}{\text{SUM_SAMPS}}}$$

2.18 Three Current Vector Sum

Not supported.

2.19 RMS Voltage

Table 17 Quant Compensation For Voltage - Inputs

Name	Offset	Note
QUANT_V0	0x27	Quantization compensation
QUANT_V1	0x2B	Quantization compensation

Table 18 Voltages - Outputs

Name	Offset	Note
V0SQSUM_X	0x8E	Sum of Squares of Voltage Samples of V0
V1SQSUM_X	0x8F	Sum of Squares of Voltage Samples of V0

$$VnSQSUM_X' = VnSQSUM_X + \frac{QUANT_Vn * SUM_SAMPS}{2^{14}}$$

$$VRMS_n = VMAX * \sqrt{\frac{C_{VSQ} * VnSQSUM_X}{SUM_SAMPS}}$$

2.20 Sensor Selection

This CE firmware supports using resistor divider for non-isolated sensing of line voltage and isolated sensing of line current with either two current transformers (CT), Shunt/CT or SY7M017 remotes. When using SY7M017 remotes, set the 6K_TEMP bit in CECONFIG[23] to allow independent control of temperature compensation adjustment variables GAIN_ADJ1-GAIN_ADJ4 by the MPU. The default configuration for the CE firmware is supporting two CTs or Shunt/CT.

2.21 SAG Detection

The CE code allows for Sag detection on the incoming voltage signal selected by FREQSEL bits. The sag detection monitors the individual samples and if they are below SAG_THR for SAG_CNT samples then sag is triggered.

Table 19 SAG Configuration - Inputs

Name	Offset	Note
CECONFIG[20]	0x20	YPULSE Interrupt tied to sag detection if 1
CECONFIG [19:8]	0x20	SAG_CNT: Number of samples voltage must be less than sag threshold to indicate sag event.
SAG_THR	0x24	Sag voltage threshold

$$\text{Peak Voltage Sag Threshold} = VMAX * C_V * SAG_THR$$

Table 20 SAG Status - Outputs

Name	Offset	Note
CESTATUS_X[0]	0x80	Set to 1 if sag detected on VA

If configured through the CECONFIG then the CE will attempt to cause an MPU interrupt on YPULSE in the event of sag detection.

2.22 Harmonic Analysis

Input and output registers available from Compute Engine to perform and analyze harmonics present on the power measurements are presented below.

Table 21 NFREQ - Inputs

Name	Offset	Note
NFREQ	0x54	Band-pass filter (BPF) center frequency

Table 22 Harmonics - Outputs

Name	Offset	Note
W0SUM_H_X	0x93	Sum of (Current * Voltage) Samples (I0*V0) at BPF center frequency
W1SUM_H_X	0x94	Sum of (Current * Voltage) Samples (I1*V0) at BPF center frequency
WSUM_H_X	0x92	Total accumulation = W0SUM_H_X for EQU=0,1
I0SQSUM_H_X.H	0x95	Sum of Squares of Current Samples high bits (I0) at BPF center frequency
I0SQSUM_H_X.L	0x96	Sum of Squares of Current Samples low bits (I0) at BPF center frequency
I1SQSUM_H_X.H	0x97	Sum of Squares of Current Samples high bits (I1) at BPF center frequency
I1SQSUM_H_X.L	0x98	Sum of Squares of Current Samples low bits (I1) at BPF center frequency
V0SQSUM_H_X	0x99	Sum of Squares of Voltage Samples (V0) at BPF center frequency
V1SQSUM_H_X	0x9A	Sum of Squares of Voltage Samples (V0) at BPF center frequency

The Compute Engine measures harmonic contents on voltage and current using band-pass filter whose center frequency is programmable thru the *NFREQ* (0x54) register. When *NFREQ* < 0 the compute engine tunes the band-pass filter to the fundamental frequency and provides the watt-hour (*WSUM_H_X*) data for fundamental component only.

Following is the formula for computing *NFREQ* register values for harmonic components.

$$NFREQ = 2^{14} \sqrt{(2 - 2^{-9})(1 - \cos(\frac{2\pi f_{pk}}{F_S}))}$$

Where

f_{pk} = BPF center frequency in Hz

F_S = 4096 Hz = sample rate

The CE provides fundamental frequency measurements of reactive energy using a dedicated band-pass filter. This can be used to determine rms fundamental currents at low currents where rms current is noisy using the following formula.

$$IxSUM_F_X = \sqrt{\frac{WxSUM_F_X^2 + VARxSUM_F_X^2}{VxSQSUM_F_X}}$$

where

WxSUM_F_X is WxSUM_H_X with NFREQ=-1

VxSQSUM_F_X is VxSQSUM_H_X with NFREQ=-1

Table 23 Fundamental - Outputs

Name	Offset	Note
VAR0SUM_F_X	0xA5	Sum of (Current * 90°shifted Voltage) Samples at BPF center frequency (fundamental frequency)
VAR1SUM_F_X	0xA6	Sum of (Current * 90°shifted Voltage) Samples at BPF center frequency (fundamental frequency)
VARSUM_F_X	0xA4	Total accumulation = VAR0SUM_F_X for EQU=0,1

2.23 Voltage Phase Angle

Not supported.

2.24 DC Measurements

Not supported.

2.25 Chip ID

The CE reports chip ID value in CHIPID_X register.

Table 24 Chip ID Outputs

Name	Offset	Note
CHIPID_X	0x90	Chip ID value

2.26 VLS Mode

To put the SY7T M11 into VLS mode, the MPU must set the ADC_VLS bit in the ADC.CNTL register (bit 24), the RFS bit in CECONFIG (bit 29), and SUM_SAMPS in the CE.CNTL register changed to 324. The sample rate is reduced to 324.44Hz in this mode and only IA and IB have valid measurements. The RFS bit steers a secondary path for IA that was running in the background in mission mode to the IOSQSUM_X register. No external adjusts by the MPU are required because the adc gain difference is adjusted internally by the ce code. A similar operation is performed on the IB path to the I1SQSUM_X, but no gain adjustment is required.

3 CE Map

The CE operates within a bounded RAM space starting at a fixed location. This offset (in the MPU code) is shown in the example code below.

```
#define CE_DATA_RAM 0x2000_0000
#define ce_ram ((int32_t *) (CE_DATA_RAM))
```

Starting at an offset of 0 (from the CE_DATA_RAM location), the CE firmware uses this RAM as shown in the table below. The CE RAM addresses its RAM as 32bit (4-byte) words. It considers its RAM starting point to be a 0 offset and all RAM addresses grow up from that in 4-byte increments.

Table 25 CE Data RAM

Start	Finish	Note
0x0000	0x0006	Raw ADC values
0x0010	0x005F	Input Parameters
0x0060	0x007F	Working Variables
0x0080	0x00A6	Measurement Outputs
0x00A7	0x00FF	Working Variables

3.1 Input Parameter Map

Default variable values are set per the CT case. Unsigned parameters are marked with a † symbol, registers containing bit patterns that do not represent numbers are marked with two † symbols.

Table 26 CE RAM - Inputs

Name	Offset	Default	Note
I0_RAW	0x00	0x00000000	
V0_RAW	0x02	0x00000000	
I1_RAW	0x03	0x00000000	
CAL_I0	0x10	0x00004000	
CAL_V0	0x11	0x00004000	
PHADJ_0	0x12	0x00000000	
CAL_I1	0x13	0x00004000	
CAL_V1	0x14	0x00004000	
PHADJ_1	0x15	0x00000000	
I1_SCALE	0x16	0x00004000	
IBSCALE†	0x17	0x00000001	

Name	Offset	Default	Note
CECONFIG ^{††}	0x20	0x00316C08	
CECONFIG.0		0	PULSE_SLOW
CECONFIG.1		0	PULSE_FAST
CECONFIG.2		0	VG4X
CECONFIG.3		1	INT_CREEP
CECONFIG.4		0	IG8X
CECONFIG.5		0	EXT_PULSE
CECONFIG[7:6]		00	FREQSEL
CECONFIG[19:8]		0x16C	SAG_CNT
CECONFIG.20		1	SAG_INT
CECONFIG.21		1	EDGE_INT
CECONFIG.22		0	EXT_TEMP
CECONFIG.29		0	RFS
WRATE [†]	0x21	10346	
KVAR [†]	0x22	6445	
CE_SUMPRE [†]	0x23	4096	
SAG_THR [†]	0x24	0x1D11BBE	
CREEP_THR [†]	0x25	1377	
NFREQ	0x54	-1	
QUANT_V0	0x27	0x00000000	
QUANT_I0	0x28	0x00000000	
QUANT_W0	0x29	0x00000000	
QUANT_VAR0	0x2A	0x00000000	
QUANT_V1	0x2B	0x00000000	
QUANT_I1	0x2C	0x00000000	
QUANT_W1	0x2D	0x00000000	
QUANT_VAR1	0x2E	0x00000000	
GAIN_ADJ0	0x55	0x00004000	
GAIN_ADJ1	0x56	0x00004000	
GAIN_ADJ2	0x57	0x00004000	
APULSEW	0x58	0x00000000	
WPULSE_CTR [†]	0x59	0x00000000	

Name	Offset	Default	Note
WPULSE_FRAC†	0x5A	0x00000000	
WSUM_ACCUM	0x5B	0x40000000	
APULSER	0x5C	0x00000000	
VPULSE_CTR†	0x5D	0x00000000	
VPULSE_FRAC†	0x5E	0x00000000	
VSUM_ACCUM	0x5F	0x40000000	

3.2 Measurement Output Map

Table 27 CE RAM - Outputs

Name	Offset
CESTATUS_X††	0x80
TEMP_X	0x81
FREQ_X†	0x82
MAINEDGE_X†	0x83
WSUM_X	0x84
W0SUM_X	0x85
W1SUM_X	0x86
VARSUM_X	0x87
VAR0SUM_X	0x88
VAR1SUM_X	0x89
I0SQSUM_X.H	0x8A
I0SQSUM_X.L†	0x8B
I1SQSUM_X.H	0x8C
I1SQSUM_X.L†	0x8D
V0SQSUM_X	0x8E
V1SQSUM_X	0x8F
WSUM_H_X	0x92
W0SUM_H_X	0x93
W1SUM_H_X	0x94
I0SQSUM_H_X.H	0x95

Name	Offset
I0SQSUM_H_X.L†	0x96
I1SQSUM_H_X.H	0x97
I1SQSUM_H_X.L†	0x98
V0SQSUM_H_X	0x99
V1SQSUM_H_X	0x9A
VARSUM_F_X	0xA4
VAR0SUM_F_X	0xA5
VAR1SUM_F_X	0xA6

4 Calculation Constants

This document uses several constants/symbols to describe the operation of the CE. The following table gives values/descriptions for these values.

Table 28 CE Constants

Name	Value	Units	Note
FS	4096	samples/sec	ADC sampling rate
SUM_SAMP	4096	samples	Number of samples per accumulation
FSV		Volts _{pk}	Peak Voltage at which the ADC will see 0.250 V _{pk}
VMAX	$\frac{FSV}{\sqrt{2}}$	Volts _{rms}	RMS Voltage at which the ADC will see 0.1768V _{rms}
FSI		Amps _{pk}	Peak Current at which the ADC will see 0.250 V _{pk}
IMAX	$\frac{FSI}{\sqrt{2}}$	Amps _{rms}	RMS Current at which the ADC will see 0.1768V _{rms}
CvsQ	1.09435E-05	Voltage ² -samples	Voltage ² CE Scalar
CISQ	1.05753E-05	Current ² -samples	Current ² CE Scalar for Line Current (I0)
CINSQ	4.37742E-05	Current ² -samples	Current ² CE Scalar for Neutral Current (I1)
CP	1.07592E-05	Watt-samples	Power CE Scalar for Power
Cv	6.03097E-09	Volt/VMAX	Voltage CE Scalar
QUANT_V_LSB	6.67941E-10	Volt ² /VMAX ²	V0,V1,V2 QUANT LSB
QUANT_I_LSB	3.12525E-13	Amp ² /IMAX ²	I0,I1,I2,I3 QUANT LSB
QUANT_W_LSB	6.56692E-10	Watt/(VMAX × IMAX)	W0,W1,W2 QUANT LSB
QUANT_VAR_LSB	6.56692E-10	Watt/(VMAX × IMAX)	VAR0,VAR1,VAR2 QUANT LSB
CIDC	N/A	IMAX*Amps	I0,I1,I2 DC SUM LSB
CVDC	N/A	VMAX*Volts	V0,V1,V2 DC SUM LSB

4.1 Sample Structure

Use this C structure to hold functions and constants when the CE code must be dynamically switched.

This contains all the data needed to run and use a CE code. If other data is needed, add it on the end.

```
const ce_t Cem11a02f =
{
    // Code
```

```
Cem11a02f_init,  
Cem11a02f_enable,  
// LSBs  
4096,           // CE_FS, from CE's Ref. Manual's Table  
4096,           // CE_SUM_SAMP, from table  
1.09435E-05,    // CE_CVSQ, from table  
1.05753E-05,    // CE_CISQ, from table  
1.07592E-05,    // CE_CP, from table  
6.03097E-09,    // CE_CV, from table  
6.56692E-10,    // CE_QUANT_W_LSB, from table  
3.12525E-13,    // CE_QUANT_I_LSB, from table  
6.67941E-10,    // CE_QUANT_V_LSB, from table
```

5 Setting the MPU Processor

5.1 MPU

The CE operates as a slave to the MPU processor. It requires the MPU properly configures and starts the CE/AFE. The MPU core should configure data acquisition system as follows.

Table 29 MPU Core Configurations for Use with two CTs or Shunt/CT

Name	MPU Address	Value	Note
CE.PLS_CNTL	0x4000_D010	0x00F70067	PLS_INTERVAL=103, PLS_WID_MAX=248
CE.INT_CNTL	0x4000_D018		Value set per MPU (MPU) application needs
CE.CNTL	0x4000_D014	0x00631000	CE.CNTL[12:0] = SUM_SAMPS = 4096 CE.CNTL[16] = CE_EN = 1 CE.CNTL[17] = RTM_EN = 1 CE.CNTL[19:18] = CHOP_EN = 0x00b CE.CNTL[20] = FCHOP_EN = 0 CE.CNTL[21] = PRECHOP_EN = 1 CE.CNTL[22] = BGCHOP_EN = 1 CE.CNTL[26:24] = EQU = 0x0 (or 0,1) CE.CNTL[28] = INVERT_PLS = 0
ADC.VREF_CNTL	0x4000_D020	0x00010003	CE.VREF_CNTL[0] = ADCS_EN = 1 CE.VREF_CNTL[1] = ADCM_EN = 1 CE.VREF_CNTL[8] = PRE_EN = 0 CE.VREF_CNTL[16] = VREF_EN = 0 CE.VREF_CNTL[29:24] = FIRSTR = 0x0
ADC.CNTL	0x4000_D024	0x00005325	CNTL[1:0] = ADCS_SPD = 1 CNTL[3:2] = ADCM_SPD = 1 CNTL[5:4] = MUX_DIV = 2 CNTL[11:8] = ADCS_BIAS = 3 CNTL[15:12] = ADCM_BIAS = 5

Name	MPU Address	Value	Note
			CNTL[16] = LOW_BIAS = 0 CNTL[24] = ADC_VLS = 0
ADC.SLOT_CNTL	0x4000_D028	0x00000021	ADC.SLOT_CNTL[1:0] = SLOT0_SEL = 1 ADC.SLOT_CNTL[5:4] = SLOT1_SEL = 2
ADC.FIR_LEN	0x4000_D02C	0x000000BB	ADC.FIR_LEN[3:0] = FIRO_LEN = 0xB (288) ADC.FIR_LEN[7:4] = FIRO_LEN = 0xB (288)
S_LEN		0x00000000	
RMT.CNTL	0x4000_9000	0x00000000	RMT.CNTL[4:0] = SIDE_SEL = 0x0 RMT.CNTL[5] = RS_CMTEST = 0 RMT.CNTL[6] = RX_CAPTEST = 0 RMT.CNTL[8] = XTRA_10P = 0 RMT.CNTL[9] = R2K_EN = 0 RMT.CNTL[10] = SWAP = 0 RMT.CNTL[16] = PAD_EN = 0 RMT.CNTL[20] = CLK_EN = 0
RMT.CAP_BALA	0x4000_9030		RMT.CAP_BALA[4:8] = 0x00 RMT.CAP_BALA[7:5] = 0x0 RMT.CAP_BALA[9:8] = 0x0
RMT.CAP_BALB	0x4000_9034		RMT.CAP_BALB[4:8] = 0x00 RMT.CAP_BALB[7:5] = 0x0 RMT.CAP_BALB[9:8] = 0x0
RMT.CAP_BALC	0x4000_9038		RMT.CAP_BALC[4:8] = 0x00 RMT.CAP_BALC[7:5] = 0x0 RMT.CAP_BALC[9:8] = 0x0

Table 30 MPU Core Configurations for Use with two CTs or Shunt/CT in Low Power Mode

Name	MPU Address	Value	Note
CE.PLS_CNTL	0x4000_D010	0x00F70067	PLS_INTERVAL=103 , PLS_WID_MAX=248
CE.INT_CNTL	0x4000_D018		Value set per MPU (MPU) application needs
CE.CNTL	0x4000_D014	0x00630144	CE.CNTL[12:0] = SUM_SAMPS = 324 CE.CNTL[16] = CE_EN = 1 CE.CNTL[17] = RTM_EN = 1 CE.CNTL[19:18] = CHOP_EN = 0x00b CE.CNTL[20] = FCHOP_EN = 0 CE.CNTL[21] = PRECHOP_EN = 1 CE.CNTL[22] = BGCHOP_EN = 1 CE.CNTL[26:24] = EQU = 0x0 (or 0,1) CE.CNTL[28] = INVERT_PLS = 0
ADC.VREF_CNTL	0x4000_D020	0x00010003	CE.VREF_CNTL[0] = ADCS_EN = 1 CE.VREF_CNTL[1] = ADCM_EN = 1 CE.VREF_CNTL[8] = PRE_EN = 0 CE.VREF_CNTL[16] = VREF_EN = 1 CE.VREF_CNTL[29:24] = FIRSTR = 0x0
ADC.CNTL	0x4000_D024	0x01005325	CNTL[1:0] = ADCS_SPD = 1 CNTL[3:2] = ADCM_SPD = 1 CNTL[5:4] = MUX_DIV = 2 CNTL[11:8] = ADCS_BIAS = 5 CNTL[15:12] = ADCM_BIAS = 3 CNTL[16] = LOW_BIAS = 0 CNTL[24] = ADC_VLS = 1
ADC.SLOT_CNTL	0x4000_D028	0x00000021	ADC.SLOT_CNTL[1:0] = SLOT0_SEL = 1 ADC.SLOT_CNTL[5:4] = SLOT1_SEL = 2

Name	MPU Address	Value	Note
ADC.FIR_LEN	0x4000_D02C	0x000000BB	ADC.FIR_LEN[3:0] = FIRO_LEN = 0xB (288) ADC.FIR_LEN[7:4] = FIRO_LEN = 0xB (288)
S_LEN		0x00000000	
RMT.CNTL	0x4000_9000	0x00000000	RMT.CNTL[4:0] = SIDE_SEL = 0x0 RMT.CNTL[5] = RS_CMTEST = 0 RMT.CNTL[6] = RX_CAPTEST = 0 RMT.CNTL[8] = XTRA_10P = 0 RMT.CNTL[9] = R2K_EN = 0 RMT.CNTL[10] = SWAP = 0 RMT.CNTL[16] = PAD_EN = 0 RMT.CNTL[20] = CLK_EN = 0
RMT.CAP_BALA	0x4000_9030		RMT.CAP_BALA[4:8] = 0x00 RMT.CAP_BALA[7:5] = 0x0 RMT.CAP_BALA[9:8] = 0x0
RMT.CAP_BALB	0x4000_9034		RMT.CAP_BALB[4:8] = 0x00 RMT.CAP_BALB[7:5] = 0x0 RMT.CAP_BALB[9:8] = 0x0
RMT.CAP_BALC	0x4000_9038		RMT.CAP_BALC[4:8] = 0x00 RMT.CAP_BALC[7:5] = 0x0 RMT.CAP_BALC[9:8] = 0x0

5.2 The CE Init and Start Function

These are models for setting up the electronics for this CE code. The register values are exact, but the connections to the higher-level application code should be changed as needed.

The init function sets-up the CE and front-end hardware. After this function, the MPU code should install calibration data into the CE RAM, and then call the start function below. The init function should not start the CE or the ADCs.

```
void Cem11a02f_init(void)
```

```
{  
    // Enable the pulses after glitches are done.  
    // Note that these operations intentionally fail  
    // if the pin is an LCD pin.  
    gpio_set_state(wpulse_seg,1);  
    gpio_dir_out(wpulse_seg);  
    gpio_alt_on(wpulse_seg);  
  
    gpio_set_state(vpulse_seg,1);  
    gpio_dir_out(vpulse_seg);  
    gpio_alt_on(vpulse_seg);  
  
    gpio_set_state(xpulse_seg,1);  
    gpio_dir_out(xpulse_seg);  
    gpio_alt_on(xpulse_seg);  
  
    gpio_set_state(ypulse_seg,1);  
    gpio_dir_out(ypulse_seg);  
    gpio_alt_on(ypulse_seg);  
  
    // Initialize CE , ADC registers :  
    CE->CNTL_b.ce_en = 0; // Disable CE  
    // Add code for a delay for 2 sample times here.  
    delay(DELAY_US(488)); // 2/4096 S  
    ADC->CNTL = 0; // Set ADC clocks off.  
    // Add code for a delay for 1 sample time here.  
    delay(DELAY_US(245)); // 1/4096 S  
    CE->INT_CNTL = 0; // Disable and clear all CE interrupts.  
    CE->PLS_CNTL_b.pls_interval = 0x68; // Set pulse interval  
    CE->PLS_CNTL_b.pls_wid_max = 0x66; // Set the maximum pulse width  
    CE->CNTL_b.sum_samps = 0x1000; // Set the sum samples  
    CE->CNTL_b.chop_en = 0; // Enable default chop mode  
    CE->CNTL_b.fchop_en = 0; // Enable fast chop  
    CE->CNTL_b.prechop_en = 1; // Enable preamp's chop  
    CE->CNTL_b.bgchop_en = 1; // Enable bandgap's chop  
    CE->CNTL_b.equ = EQUATION; // Set the metering equation (for CE code)
```

```

CE->CNTL_b.invert_pls = 0; // Do not invert pulses (0: high = off)
// Set up the ADCs.
    ADC->VREF_CNTL_b.adcs_en = 1; // ADCS (Staring) clock enable
    ADC->VREF_CNTL_b.adcm_en = 1; // ADCM (Multiplexed) clock enable
ADC->VREF_CNTL_b.pre_en = 0; // Preamp enabled.
ADC->VREF_CNTL_b.fir_str = 0; // Enable fir-length stretch for slot 0.
ADC->CNTL_b.adcs_bias = 0x03; // Set the bias voltage of the staring ADC(s)
ADC->CNTL_b.adcm_bias = 0x05; // Set the bias voltage of the multiplexed ADC
ADC->CNTL_b.low_bias = 0; // Set the bias range
ADC->SLOT_CNTL_b.slot0_sel = 0x01; // The ADC1 input for slot 0.
ADC->SLOT_CNTL_b.slot1_sel = 0x02; // The ADC1 input for slot 1.
ADC->FIR_LEN_b.fir0_len = 0xB; // The FIR length for slot 0.
ADC->FIR_LEN_b.fir1_len = 0xB; // The FIR length for slot 1.
    RMT->CNTL = 0; // No remote for this CE
// Load the CE's data and code.
ce_load_code(&CeCode[0]);
ce_load_ram(&CeData[0], NumCeData);
// Enable needed CE interrupts; Occurs elsewhere in code.
}

```

This function loads the CE's RAM with starting data. It is called from the init function above.

It can be shared by several CE code initialization functions.

```

void load_ce_data(int *ce_data_ptr, int ary_len)
{
    int len;
    int *ce_ram_ptr;
    // Clear CE RAM Data
    len = 256;          // 1 KB CE RAM
    ce_ram_ptr = (uint32_t *) CE_RAM; // Destination pointer
    while(len > 0)
    {
        *ce_ram_ptr++ = 0x00;
        --len;
    }

    // Move CE Data image from flash to start of CERAM.
    len = ary_len; // The number of words in the 'CeData' array

```

```

        ce_data_ptr = (const unsigned int *)ce_data_ptr; // Source Pointer
        ce_ram_ptr = (uint32_t *) CE_RAM; // Destination pointer
        while(len > 0)
        {
            *ce_ram_ptr++ = *ce_data_ptr++;
            --len;
        }
    } void ce_load_ram(const unsigned int * ce_data_ptr, int len_in)
    {
        int len;
        int index = 0;
        uint32_t *ce_ram_ptr;    // Destination ptr

        // Clear CE RAM Data
        len = 256;    // 1 KB CE RAM
        ce_ram_ptr = (uint32_t *) CE_RAM; // Destination pointer
        while(len > 0)
        {
            *ce_ram_ptr++ = 0x00 ;
            --len;
        }

        // Move CE Data image from flash to start of CERAM.
        // CE data ends at higher address.
        len = len_in; // The number of words in the 'CeData' array

        ce_ram_ptr = (uint32_t *) CE_RAM; // Destination pointer
        while(len > 0)
        {
            *ce_ram_ptr++ = *ce_data_ptr++;
            --len;
        }

        ce_ram_ptr = (uint32_t *) CE_RAM; // CE RAM base
        while(index < CE_NAME_SZ)
        { // Make copy of CE name, CE will use CE_RAM & overwrite it when it starts

```

```

        // CE name is in big-endian, change it to little-endian with __rev()
        ce_ver[index++] = __rev(*(ce_ram_ptr + ce_ptr->name_idx));
        ce_ram_ptr++;
    }
}

```

The start function starts the CE and front-end hardware. It should run after the init function above, and after the MPU code installs calibration data into the CE RAM. This function starts the CE and the ADCs.

After start, the CE data is not valid for two (2) accumulation intervals.

Pulses and pulse interrupts should be enabled only **after** the CE data is valid.

The CE interrupts should be enabled when the software design requires it.

```

void Cem11a02f_enable(void)
{
    CE->CNTL_b.ce_en = 1; // Enable CE (The CE does not run until the ADC clock starts)
    ADC->CNTL_b.adc_vls = 0; // 1=Slow the system PLL by a factor of 13.
    ADC->CNTL_b.adcs_spd = 1; // Set staring ADC clock speed
    ADC->CNTL_b.adcm_spd = 1; // Set multiplexed ADC clock speed
    // Start the analog front end:
    ADC->CNTL_b.mux_div = 2; // Set the number of ADC slots per mux. frame
    // Note that the vls bit affects the CPU system clock, including the peripheral clocks.
    // Peripherals should be set-up using the final clock rate.
}

```

6 CE Resource Usage

6.1 CE Code Flash Size

The CE code image is an array of unsigned char (1 byte) 2914 entries long. The file is formatted as a constant C array of unsigned char.

```
Const __code uint32_t NumCeCode=1457; // The number of 16-bit
words in the 'CeCode' array.
Const __code uint8_t CeCode[2914]={..}
```

6.2 CE Data Flash Size

The CE initial data image is an array of unsigned char (1 byte) 1860 entries long. The file is formatted as a constant C array of unsigned char.

```
Const __code uint32_t NumCeData=465; // The number of
32-bit words in the 'CeData' array.
Const __code uint8_t CeData[1860]={..}
```

6.3 CE Cycles Per Pass

The CE runs a program pass of a fixed length per set of ADC samples. The MPU must configure the CE in such a way as to allow the CE program to complete on every set of ADC samples.

This ce code uses 2059 CE cycles out of 2400 CE cycles available.

7 CE Operational Considerations

7.1 CE Validation at Power Up

After power up, the MPU should ignore the data from the first 2 accumulation periods signaled by XFER_BUSY to allow initialization of the CE.

7.2 Filtering Out Unwanted Pulses At Start Up

To prevent unwanted pulses at start up or while in CREEP:

At start up, before initializing the CE, program the pulse pins to be DIOs that do not light the pulse LEDs. On the SY7T M11, for example, the WPULSE is output to DIO42. Set the LCD.DIO42(0x4000_C2A8) = 0x03. When the CE's data is reliable, reprogram the pins as pulse outputs by setting the DIO field to Alternate function and as an output pin (=0x06). Here is the table listing the settings of the 2 pulses:

	Shared with	LCD_MAPL (0x4000_C00C)	LCD_MAPH (0x4000_C010)	LCD.DIOx	Start-up LCD.DIOx =	Once stabled LCD.DIOx
WPULSE	DIO42	N/A	N/A	0x4000_C2A8	0x03	0x06
VPULSE	DIO41	N/A	N/A	0x4000_C2A4	0x03	0x06

While in creep, write 0x40000000 to WSUM_ACCUM and VSUM_ACCUM.

8 Example Calculations

Assuming a 1mΩ shunt is used for line current sense and a 2MΩ+750Ω voltage divider used for line voltage sense. Based upon this, we will assume the defaults for constants below.

Table 31 CE Constants For 1mΩ shunt

nameName	valueValue	unitsUnits	noteNote
SUM_SAMPS	4096	samples/accum	Default samples per accumulation interval
FS	4096	samples/sec	Default ADC sampling rate
FSV	666.92	Volts _{pk}	Peak Voltage at which the ADC will see 0.250 V _{pk}
VMAX	471.58	Volts _{rms}	RMS Voltage at which the ADC will see 0.1768V _{rms}
FSI	250A	Amps _{pk}	Peak Current at which the ADC will see 0.250 V _{pk}
IMAX	176.78	Amps _{rms}	RMS Current at which the ADC will see 0.1768V _{rms}

8.1 RMS Voltage

$$VRMS_n = VMAX * \sqrt{\frac{C_{VSQ} * VOSQSUM_X}{SUM_SAMPS}}$$

$$VRMS_n = VMAX * \sqrt{2.671753E-09 * VOSQSUM_X}$$

$$VRMS_n = VMAX * 5.1689002E-05 * \sqrt{VOSQSUM_X}$$

$$VRMS_n = 0.0243755 * \sqrt{VOSQSUM_X} \text{ Volts}$$

8.2 RMS Current

$$IRMS_n = IMAX * \sqrt{\frac{C_{ISQ} * IOSQSUM_X}{SUM_SAMPS}}$$

$$IRMS_n = IMAX * \sqrt{2.581860352E-09 * IOSQSUM_X}$$

$$IRMS_n = IMAX * 5.08120099146E-05 * \sqrt{IOSQSUM_X}$$

$$IRMS_n = 0.008982547113 * \sqrt{IOSQSUM_X} \text{ Amps}$$

8.3 Energy & Power

$$\text{Power n} = (V_{MAX} * I_{MAX}) * \left(\frac{C_P}{SUM_SAMPS} \right) * SUM_X$$

$$\text{Power n} = (V_{MAX} * I_{MAX}) * 2.6267578125E - 09 * SUM_X$$

$$\text{Power n} = 0.000218982061 * SUM_X \quad \text{Watts}$$

$$\text{Energy n} = (V_{MAX} * I_{MAX}) * \left(\frac{C_P}{FS} \right) * SUM_X$$

$$\text{Energy n} = (V_{MAX} * I_{MAX}) * 2.6267578125E - 09 * SUM_X$$

$$\text{Energy n} = 0.000218982061 * SUM_X \quad \text{Watt} \times \text{sec}$$

8.4 SAG Threshold

$$\text{Peak Voltage Sag Threshold} = V_{MAX} * C_V * SAG_THR$$

$$\text{Peak Voltage Sag Threshold} = V_{MAX} * 6.03097E - 09 * SAG_THR$$

$$\text{Peak Voltage Sag Threshold} = 0.000002844085 * SAG_THR \quad \text{Volts}$$

9 Using QUANT to Compensate for Nonlinearities

9.1 ENERGY

Nonlinearity is most noticeable at low currents, as shown in Figure 6, and can result from input noise and truncation. Nonlinearities can be eliminated using the QUANT_W0 variable.

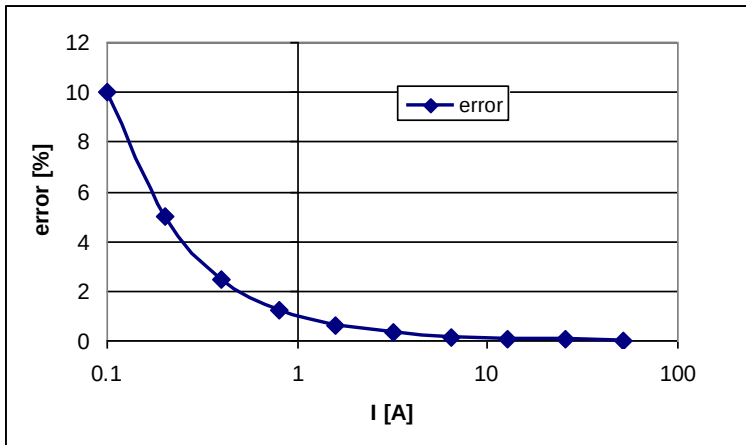


Figure 6 Nonlinearity at Low Current

The error can be seen as the presence of a virtual constant noise current. While 10mA hardly contribute any error at currents of 10A and above, the noise becomes dominant at small currents.

The value to be used for QUANT_Wn can be determined by the following formula:

$$QUANT_Wn = -\frac{\frac{error}{100} V \cdot I}{V_{MAX} \cdot I_{MAX} \cdot LSB}$$

Where error = observed error at a given voltage (V) and current (I),

VMAX = voltage scaling factor

IMAX = current scaling factor

LSB = QUANT LSB value = 6.56692×10^{-10}

Example: Assuming an observed error as in Figure 6, we determine the error at 1A to be +1%. If VMAX is 600V and IMAX = 133A, and if the measurement was taken at 240V, we determine QUANT_Wn as follows:

$$QUANT_Wn = -\frac{\frac{1}{100} 240 \cdot 1}{600 \cdot 133 \cdot 6.56692 \cdot 10^{-10}} = -45798$$

It does not matter which current value is chosen as long as the corresponding error value is significant (5% error at 0.2A used in the above equation will produce the same result for QUANT_Wn).

Input noise and truncation can cause similar errors in the VAR calculation that can be eliminated using the QUANT_VARn variables. QUANT_VARn is determined using the same formula as QUANT_Wn.

9.2 CURRENT

Calculating QUANT_I0 is similar to the calculations for energy except as follows.

$$QUANT_In = -\frac{\frac{2 \cdot error}{100} I \cdot I}{IMAX \cdot IMAX \cdot LSB}$$

Where error = observed error at a current (I)

IMAX = current scaling factor

LSB = QUANT LSB value = 3.12525×10^{-13}

9.3 VOLTAGE

Calculating QUANT_V0 is similar to the calculations for energy except as follows.

$$QUANT_Vn = -\frac{\frac{2 \cdot error}{100} V \cdot V}{VMAX \cdot VMAX \cdot LSB}$$

Where error = observed error at a voltage (V)

VMAX = voltage scaling factor

LSB = QUANT LSB value = $6.67941E - 10$

.

REVISION HISTORY

Revision, Date	Change	Page(s)
1.0, December 19, 2017	Initial Draft	
1.1, March 1, 2018	Corrected Determining IBSCALE and I1SCALE Value figure	Pg. 14
1.2, July 12, 2018	Updated to ce211a02a. Updated CE Code Flash Size. Updated CE Cycles Per Pass.	All Pg. 39 Pg. 39
1.3, August 31, 2018	Updated to ce211a02b. Updated CE Code Flash Size. Updated CE Data Flash Size. Updated CE Cycles Per Pass. Changed Low Power Mode to VLS Mode.	All Pg. 39 Pg. 39 Pg. 39 Pg. 24
1.4, September 11, 2018	Updated to ce211a02c. Changed title name from SY7T211a01 to SY7T M11. Updated CE Code Flash Size. Updated CE Cycles Per Pass. Changed AM11 to SY7T M11. Changed value in Spurious Pulse Preventions to 0x40000000 and updated startup description.	All Pg. 1 Pg. 39 Pg. 39 All Pg. 40
1.5, September 12, 2018	Changed WSUM_ACCUM/VSUM_ACCUM values to 0x40000000 in CE RAM – Inputs table. Added caution note in VPULSE section about dual use of VPULSE pin with SWD_RST. Added explanations on signed/unsigned types for CE registers throughout the document.	Pg.26-27 Pg. 15 All
1.6, October 16, 2018	Added C _{INSQ} to CE Constants Table. Added FSIL/FSIN ratio formula/description to Neutral Current Sensor on IB section.	Pg. 29 Pg. 12
1.7, October 25, 2018	Corrected C _{INSQ} value.	Pg. 29
1.8, January 18, 2019	Changed name from of ce file to ceM11a02d. Updated CE Code Flash Size. Updated CE Cycles Per Pass.	All Pg. 40 Pg. 40
1.9, February 4, 2019	Changed name from of ce file to ceM11a02e. Updated CE Code Flash Size. Updated CE Cycles Per Pass. Fixed NFREQ location in CE RAM - Inputs	All Pg. 40 Pg. 40 Pg. 27
2.0, January 6, 2020	Updated Figure 1. Removed Figure 2 & 6. Updated CREEP_THR descriptions. Updated PLS_INTERVAL and PLS_WID_MAX descriptions. Fixed typos and other. Updated CE RAM location.	Pg. 9 Pg. 16 Pg. 17 All Pg. 25

	Added IO_RAW, VO_RAW, and I1_RAW to CE RAM-Inputs table.	Pg. 25
	Added W1SUM_H_X to Measurement Output Map table.	Pg. 27
	Added FSI _N /FSI _L ratio formula/description to Neutral Current.	Pgs. 12-14
	Updated CE.PLS_CNTL value.	Pgs. 31, 33
	Removed polyphase voltage reference.	Pg. 10
	Added note about the ADCs in Data Acquisition section.	Pg. 10
	Removed polyphase current reference.	Pg. 10
	Added Neutral Current figure number and caption	Pg. 15
2.1, February 4, 2020	Added default value of DEGSCALE.	Pg. 18
	Added default value of EXT_TEMP.	Pg. 18
	Changed VREF_CAL to VREF_EN per HRM.	Pg. 31
2.2, May 7, 2020	Updated to cem11a02f.	All
	Updated CE Resource Usage section.	Pg. 39